Ministère de l'Enseignement
Supérieur et de la Recherche
Scientifique

Année Universitaire
2025/2026

EPI DIGITAL SCHOOL
Ecole Internationale Supérieure
Privée Polytechnique de Sousse

RAPPORT DE STAGE PROFESSIONNEL

Spécialité: .....

Intitulé

.....

Lieu du stage **Réalisé par** **Encadré par**

Dédicace

Remerciements

Résumé

Ce rapport présente la conception et la réalisation d'une application web interne dédiée à la gestion des demandes techniques. Le projet couvre l'enregistrement, la qualification, l'affectation, le traitement, la résolution et la clôture des demandes, avec une historisation des changements importants. L'architecture associe un frontend Angular, une API REST Spring Boot et une base PostgreSQL, selon une organisation modulaire séparant présentation, application, domaine et persistance. La sécurité repose sur l'authentification JWT, le contrôle d'accès fondé sur les rôles et des contrôles métier contextuels. Une assistance IA intégrée via Groq propose une catégorie et une priorité à partir du titre et de la description ; le Responsable technique conserve la décision finale. La validation s'appuie sur des tests automatisés backend et frontend, une intégration locale du parcours IA et une vérification fonctionnelle humaine. Le rapport distingue les résultats mesurés sur le périmètre V1 des limites, notamment le caractère exploratoire du benchmark IA et l'absence de portail client.

Mots-clés : génie logiciel ; API REST ; Angular ; Spring Boot ; JWT ; intelligence artificielle.

Abstract

This report presents the design and implementation of an internal web application for managing technical requests. The project covers request registration, qualification, assignment, processing, resolution and closure, with the history of significant changes. The architecture combines an Angular frontend, a Spring Boot REST API and a PostgreSQL database, using modular responsibilities for presentation, application, domain and persistence. Security relies on JWT authentication, role-based access control and contextual business checks. An AI assistance feature integrated through Groq proposes a category and a priority from the request title and description ; the Technical Manager remains responsible for the final decision. Validation combines automated backend and frontend tests, a local integration of the AI workflow and human functional verification. The report distinguishes measured results within the V1 scope from remaining limitations, including the exploratory nature of the AI benchmark and the absence of a customer portal.

Mots-clés

Sommaire

Introduction générale **1**

1 Contexte et environnement du projet **2**

1.1 Présentation de l'organisme d'accueil 3

1.2 Organisation et environnement de travail 3

1.3 Contexte et problématique du projet 3

1.4 Objectifs et périmètre de la V1 3

1.5 Démarche et travaux réalisés 4

2 Analyse et conception **5**

2.1 Acteurs et exigences 6

2.2 Règles métier et cycle de vie 6

2.3 Modélisation fonctionnelle 6

2.4 Architecture générale 7

2.5 Données, API et sécurité conçues 7

2.6 Assistance intelligente 8

3 Réalisation et validation **9**

3.1 Environnement et architecture réalisée 10

3.2 Fonctionnement des fonctionnalités clés 10

3.3 Sécurité et gestion des erreurs 10

3.4 Assistance IA Groq et validation humaine 11

3.5 Tests et résultats 11

3.6 Limites et perspectives 11

Conclusion générale **12**

Annexes **13**

Références **14**

Table des figures

Liste des tableaux

Introduction générale

Le stage porte sur la conception et la réalisation d'une application web interne destinée à organiser les demandes techniques reçues des clients. L'enjeu est de qualifier les demandes, attribuer les responsabilités, sécuriser les accès, suivre les transitions et conserver une trace des changements.

La problématique est la suivante : comment construire une solution simple à exploiter, maintenable et contrôlée pour accompagner tout le cycle de vie d'une demande, tout en conservant une décision humaine lorsque l'assistance IA est sollicitée ?

Le projet vise une architecture Angular, Spring Boot et PostgreSQL, structurée en modules et protégée par JWT et RBAC. Le rapport présente d'abord le contexte et le cadrage, puis l'analyse et la conception, avant de décrire la réalisation et la validation. Les informations administratives absentes restent signalées comme éléments à compléter.

Chapitre 1

Contexte et environnement du projet

1.1 Présentation de l'organisme d'accueil

1.2 Organisation et environnement de travail

1.3 Contexte et problématique du projet

Le projet porte sur la gestion interne des demandes techniques reçues des clients. Le besoin fonctionnel couvre leur enregistrement, leur qualification, leur affectation à un Agent technique et le suivi de leur traitement jusqu'à leur clôture ou leur annulation. Chaque demande doit être associée à un client, une catégorie, une priorité et un statut.

La problématique de conception consiste à organiser ce cycle en définissant les responsabilités de chaque rôle et les conditions de passage entre les étapes. Il s'agit notamment de distinguer la déclaration de résolution par l'Agent technique de son acceptation par le Responsable technique, tout en prévoyant l'historisation des changements importants. L'assistance IA envisagée doit s'inscrire dans ce fonctionnement sans se substituer à la décision humaine.

1.4 Objectifs et périmètre de la V1

La première version vise à réunir les fonctions nécessaires à la qualification, au traitement et au suivi des demandes techniques dans une application réservée aux utilisateurs internes. Son périmètre fonctionnel est validé et gelé. Les fonctions décrites ci-dessous constituent les exigences de cette version, dont la réalisation et la validation sont présentées au chapitre 3.

Rôles et responsabilités. Trois rôles applicatifs sont retenus. Le **Responsable technique** enregistre et qualifie les demandes, organise leur affectation et suit leur avancement. Il examine les résolutions proposées et décide de la clôture ou de la reprise du traitement. L'**Agent technique** consulte ses demandes affectées, réalise leur traitement, renseigne une solution et déclare leur résolution. L'**Administrateur** gère les comptes internes et l'attribution des rôles métier autorisés. Ce rôle ne donne pas automatiquement accès aux fonctions métier. Une même personne peut cumuler plusieurs rôles.

Le Client est une entité métier associée aux demandes, sans compte ni accès à l'application. Le périmètre prévoit la sélection d'un client existant ou la saisie de ses informations minimales pendant l'enregistrement d'une demande.

Cycle de gestion. La qualification repose sur une catégorie issue de la liste prédéfinie de la V1 et sur une priorité évaluée selon le contexte. Une demande nouvelle peut ensuite être affectée, puis prise en charge par l'Agent technique. Celui-ci renseigne le traitement et la solution avant de déclarer la demande résolue. Le Responsable technique peut accepter cette résolution et clôturer

la demande, ou la refuser pour reprendre le traitement. L'annulation est prévue aux étapes autorisées, avec un motif obligatoire. La demande est conservée et les changements importants doivent être historisés.

Assistance IA. La V1 prévoit une assistance déclenchée explicitement par le Responsable technique pour proposer uniquement une catégorie et une priorité. Le Responsable technique reste responsable de la décision finale : il valide ou modifie les suggestions. Le besoin prévoit également de maintenir la gestion manuelle lorsque l'IA est indisponible ou échoue. Ces exigences fonctionnelles ne constituent pas une implémentation du module IA.

Hors périmètre. La V1 exclut le portail et le compte Client, ainsi qu'un module autonome de gestion des clients. Elle ne prévoit ni sous-catégories, ni statut d'attente, ni système de notes ou commentaires, ni suppression fonctionnelle d'une demande. L'attribution du rôle Administrateur depuis l'interface est également exclue. Enfin, l'IA ne doit ni décider d'une catégorie ou d'une priorité sans validation humaine, ni produire un résumé automatique, ni affecter automatiquement les demandes.

1.5 Démarche et travaux réalisés

Les travaux ont commencé par la précision du besoin de gestion des demandes techniques. Le périmètre de la première version a ensuite été délimité et les responsabilités des utilisateurs ont été définies. Cette base a permis d'analyser les règles métier et les étapes de traitement d'une demande, de son enregistrement à sa clôture ou à son annulation.

L'analyse a été prolongée par la modélisation des comportements attendus, des données et de leurs relations. Ces modèles ont servi à préparer la conception générale de la solution : organisation de l'application, conservation des données, échanges entre ses parties, sécurité et place de l'assistance intelligente. Cette conception reste en cours et prépare la mise en œuvre.

La démarche a ensuite été prolongée par l'implémentation du backend et du frontend, l'intégration de l'assistance IA, les tests automatisés et la validation fonctionnelle. Les résultats et leurs limites sont présentés au chapitre 3.

Chapitre 2

Analyse et conception

Ce chapitre transforme le besoin de gestion des demandes techniques en exigences, règles métier et modèles de conception. La distinction entre conception et réalisation est maintenue : les modèles décrivent la solution retenue, tandis que le chapitre suivant vérifie ce qui a effectivement été développé.

2.1 Acteurs et exigences

Le système est utilisé par trois rôles internes. Le Responsable technique enregistre, qualifie, affecte et supervise les demandes. L'Agent technique traite les demandes qui lui sont affectées, renseigne la solution et déclare une résolution. L'Administrateur gère les comptes internes et leurs rôles métier. Le Client est une donnée métier associée à une demande, mais ne possède pas de compte dans la V1.

Les exigences fonctionnelles couvrent l'authentification, la gestion des demandes, la sélection ou la saisie minimale d'un client, l'affectation, le traitement, la résolution, la clôture, le refus de résolution, l'annulation avec motif, l'historisation et l'administration des utilisateurs. L'assistance IA propose une catégorie et une priorité ; elle ne décide jamais seule.

Les exigences non fonctionnelles concernent la sécurité des accès, la cohérence transactionnelle, la maintenabilité, la lisibilité de l'interface, l'accessibilité et la traçabilité des changements. L'API doit rester indépendante de la présentation et les autorisations métier doivent être vérifiées côté serveur.

2.2 Règles métier et cycle de vie

Une demande possède une référence générée côté serveur, un client, une catégorie, une priorité et un statut. Les transitions ne sont pas de simples changements de valeur : chaque opération vérifie le rôle, l'état courant et les conditions propres à l'action. La résolution déclarée par l'Agent est distincte de son acceptation par le Responsable technique. L'annulation exige un motif et les changements importants sont historisés atomiquement avec la modification métier.

Le diagramme d'activité du cycle de vie formalise cette progression. Il répond à la question suivante : dans quelles conditions une demande peut-elle passer de l'enregistrement à la clôture, au refus ou à l'annulation ? Il est conservé dans les sources UML validées ; son rendu sera placé dans ‘figures/uml/‘ lorsque l'outil de génération sera disponible.

2.3 Modélisation fonctionnelle

Le diagramme global de cas d'utilisation met en relation les trois rôles et les fonctions de la V1. Les cinq diagrammes de séquence détaillent les scénarios majeurs : enregistrer et qualifier une demande, affecter ou réaffecter, traiter et résoudre, examiner une résolution, et demander

une analyse IA. Ils restent centrés sur l'interaction entre l'acteur et le système, afin de compléter l'analyse fonctionnelle sans exposer prématurément les détails de code.

2.4 Architecture générale

La solution adopte une application web Angular communiquant avec une API REST Spring Boot et une base PostgreSQL. Le backend est un monolithe modulaire : les modules sont regroupés par responsabilités métier (‘request‘, ‘identity / administration‘, ‘security‘ et ‘ai‘) tout en partageant un même déploiement. Cette organisation répond au besoin de séparation des responsabilités sans introduire la complexité opérationnelle de plusieurs services indépendants.

À l'intérieur des modules, les responsabilités sont séparées entre présentation, application, domaine et persistance. Les contrôleurs exposent les échanges HTTP, les services d'application orchestrent les cas d'utilisation et les transactions, le domaine porte les invariants, et les repositories isolent l'accès aux données. Les DTO séparent les contrats d'échange des entités JPA.

Cette organisation justifie plusieurs principes SOLID, mais pas tous au même niveau. La séparation des responsabilités illustre principalement le principe de responsabilité unique. ‘AnalyseIaPort‘ et son adaptateur Groq illustrent une inversion de dépendance : le cœur applicatif dépend d'une abstraction, non du fournisseur. Les autres principes ne sont pas revendiqués globalement sans preuve spécifique ; l'architecture reste une organisation pragmatique, adaptée à une V1.

2.5 Données, API et sécurité conçues

Les entités persistées sont ‘Utilisateur‘, ‘Client‘, ‘DemandeTechnique‘ et ‘HistoriqueDemande‘. JPA/Hibernate assure le mapping objet-relationnel et Spring Data JPA fournit les opérations de persistance. PostgreSQL porte les contraintes d'unicité, les relations et les instants temporels. Flyway versionne l'évolution du schéma.

L'API REST expose des ressources et des actions métier explicites. Les réponses utilisent des DTO afin de ne pas exposer directement les entités persistantes. Les modifications métier et leur historique sont regroupées dans une même transaction.

L'authentification utilise un JSON Web Token (JWT), c'est-à-dire un format compact contenant des claims, conformément à la norme correspondante (Jones, Bradley et Sakimura 2015). Angular conserve la session en mémoire et dans ‘sessionStorage‘ sous ‘safyron.session.v1‘, puis envoie le token dans l'en-tête ‘Authorization‘. Cette restauration locale ne remplace pas la vérification backend de la signature, de l'état actif, des rôles actuels et des droits. Le contrôle d'accès fondé sur les rôles (RBAC) est complété par des contrôles métier contextuels.

2.6 Assistance intelligente

L'intégration IA est conçue autour de 'AnalyseIaPort' et d'un adaptateur Groq. Le Responsable technique déclenche explicitement l'analyse à partir du titre et de la description. La réponse propose une catégorie et une priorité, qui restent modifiables et ne sont appliquées qu'après une action humaine. Une indisponibilité de l'IA doit laisser la qualification manuelle disponible.

Cette frontière permet de remplacer le fournisseur sans faire dépendre les règles métier de son SDK. Elle ne transforme pas la suggestion en décision automatique : la validation humaine reste une règle fonctionnelle et une limite importante du module.

Chapitre 3

Réalisation et validation

3.1 Environnement et architecture réalisée

La réalisation associe un frontend Angular 22 et Angular Material 22 à un backend Spring Boot. Les données sont persistées dans PostgreSQL avec JPA/Hibernate et les migrations sont gérées par Flyway. Le frontend est organisé par fonctionnalités, avec des responsabilités communes dans ‘core‘ et ‘shared‘.

Le frontend utilise des services API par fonctionnalité, des formulaires réactifs, des guards pour guider la navigation et un interceptor pour transmettre le JWT. Les guards améliorent l’expérience utilisateur, mais le backend reste l’autorité pour l’autorisation.

3.2 Fonctionnement des fonctionnalités clés

La création d’une demande permet de sélectionner un client existant ou de renseigner les informations minimales d’un nouveau client. La liste propose filtres, tri et pagination côté serveur. Le détail présente les actions permises par le rôle et l’état courant. Le workflow couvre l’affectation, la prise en charge, le traitement, la résolution, l’acceptation, le refus, la clôture et l’annulation.

Les interfaces d’administration permettent la liste paginée, la création, la modification du nom et de l’email, l’activation ou la désactivation et le remplacement des rôles métier. L’auto-désactivation de l’administrateur connecté est refusée côté serveur et signalée dans l’interface.

Les captures présentes dans ‘figures/screenshots/‘ illustrent les écrans réels de connexion, liste, création, assistance IA, sélection de client, affectation, traitement, résolution, clôture et administration. Elles seront insérées après contrôle de leur résolution et de leur lisibilité dans le PDF.

3.3 Sécurité et gestion des erreurs

Le backend vérifie le JWT, relit l’utilisateur et son état actif, puis applique le RBAC et les contrôles contextuels. Une réponse 401 termine la session Angular ; une réponse 403 signale une action interdite sans confondre l’absence d’authentification et l’absence de permission. Les mots de passe sont hachés par ‘PasswordEncoder‘ et les secrets restent externalisés (Spring 2026b ; Spring 2026a).

Les règles métier produisent des erreurs explicites, notamment lorsqu’une transition est impossible, lorsqu’une ressource n’existe pas ou lorsqu’une opération sensible est interdite. Les transactions garantissent que l’évolution d’une demande et son historique ne sont pas séparés.

3.4 Assistance IA Groq et validation humaine

Le module IA V1 est intégré pour le seul rôle de Responsable technique. Le chemin réel est Angular, Spring Boot, puis l'adaptateur Groq. Le fournisseur retourne une proposition temporaire ; aucune persistance automatique et aucune affectation automatique ne sont réalisées. La modification du titre ou de la description invalide la suggestion et impose une nouvelle analyse volontaire.

Le benchmark exploratoire documenté porte sur 24 exemples : 24 sorties valides, 23 catégories correctes et 22 priorités correctes, soit respectivement 95,8 % et 91,7 %. Ces chiffres décrivent cet échantillon et cette procédure ; ils ne constituent ni une mesure générale de performance ni une comparaison définitive avec Gemini, dont l'évaluation était incomplète. La décision finale reste humaine.

3.5 Tests et résultats

La validation backend a atteint 270 tests réussis avec ‘clean test‘ et ‘clean verify‘, sans échec, erreur ou test ignoré. Les tests hors ligne de l'IA utilisent H2 et un stub HTTP local ; ils ne doivent pas être confondus avec un appel Groq. Une intégration locale Angular vers Spring Boot et Groq a toutefois été vérifiée manuellement, ainsi que le scénario 503 sans clé, avec maintien de la qualification manuelle.

La validation frontend a atteint 184 tests réussis dans 15 fichiers, avec ‘npm run build‘ et ‘npm test – –watch=false‘. La validation humaine a couvert les workflows de demandes, l'administration, plusieurs tailles d'écran et les fonctions d'accessibilité prévues. Ces éléments distinguent les tests automatisés, l'intégration réelle et la validation fonctionnelle humaine.

3.6 Limites et perspectives

La V1 reste limitée aux demandes internes et aux rôles définis. Le client n'a pas de portail, l'IA ne produit ni résumé ni décision automatique, et le benchmark reste exploratoire. Les perspectives peuvent concerner l'élargissement du périmètre fonctionnel, une évaluation IA sur un corpus plus large et représentatif, ainsi qu'une industrialisation du déploiement, sous réserve de besoins et de validations supplémentaires.

Conclusion générale

Le stage a conduit à une application V1 couvrant l'enregistrement, la qualification, l'affectation, le traitement et le suivi des demandes techniques. La séparation entre interface Angular, API Spring Boot et persistance PostgreSQL fournit une base lisible et évolutive. La sécurité combine authentification JWT, rôles métier et contrôles contextuels.

L'assistance IA facultative via Groq propose une catégorie et une priorité, mais la décision finale reste modifiable et validée par le Responsable technique. Les tests automatisés et les validations humaines montrent que le périmètre V1 est fonctionnel dans les conditions vérifiées.

Les limites concernent le périmètre interne, l'absence de portail client et le caractère exploratoire du benchmark IA. Des travaux ultérieurs pourraient élargir le corpus d'évaluation, renforcer l'observabilité et étudier les besoins de déploiement, sous réserve de validations supplémentaires.

Annexes

Références

Jones, Michael, John Bradley et Nat Sakimura (mai 2015). *JSON Web Token (JWT)*. url : https://www.rfc-editor.org/rfc/rfc7519.html (visité le 11/10/2026).

Spring (2026a). *Externalize the Configuration of SpringApplication*. url : https://docs.spring.io/spring-boot/how-to/properties-and-configuration.html (visité le 11/10/2026).

— (2026b). *Password Storage*. url : https://docs.spring.io/spring-security/reference/7.0/features/authentication/password-storage.html (visité le 11/10/2026).

Table des matières détaillée

Introduction générale **1**

1 Contexte et environnement du projet **2**
1.1 Présentation de l'organisme d'accueil 3
1.2 Organisation et environnement de travail 3
1.3 Contexte et problématique du projet 3
1.4 Objectifs et périmètre de la V1 3
1.5 Démarche et travaux réalisés 4

2 Analyse et conception **5**
2.1 Acteurs et exigences 6
2.2 Règles métier et cycle de vie 6
2.3 Modélisation fonctionnelle 6
2.4 Architecture générale 7
2.5 Données, API et sécurité conçues 7
2.6 Assistance intelligente 8

3 Réalisation et validation **9**
3.1 Environnement et architecture réalisée 10
3.2 Fonctionnement des fonctionnalités clés 10
3.3 Sécurité et gestion des erreurs 10
3.4 Assistance IA Groq et validation humaine 11
3.5 Tests et résultats 11
3.6 Limites et perspectives 11

Conclusion générale **12**

Annexes **13**

Références **14**